

Card 1: App-to-Sidecar Bridge

Applications communicate with Dapr Sidecar via localhost (HTTP 3500, gRPC 50001), decoupling application logic from infrastructure.

Card 5: Seamless Integration

Automatically injects Sidecars in Kubernetes Pods using annotations, supporting legacy and modern runtimes.

Card 9: Distributed Lock API

Exposes lock endpoints to lease resource keys with TTLs, standardizing lock implementations without SDKs.

Card 13: Subscription Registry

Supports declarative components or dynamic scanners to push events to applications via HTTP POST calls.

Card 2: API Abstraction

Simplifies microservice operations into uniform APIs, allowing swap of databases or brokers without code changes.

Card 6: Pluggable State Stores

Loads Redis, PostgreSQL, or CosmosDB dynamically via component YAML metadata for portability across environments.

Card 10: State TTL Expiry

Employs ttlInSeconds metadata to assign temporary expiry limits to cached state values for automatic purging.

Card 14: Retry Backoff Delivery

Retries event delivery with exponential backoff if applications return errors, preventing request snowballs.

Card 3: Port Lifecycle Hook

Dapr sidecars run in the same Pod, blocking requests until application ports are fully initialized to prevent drops.

Card 7: ETag Optimistic Lock

Reads return a unique ETag; state writes check this version to prevent concurrent modifications with a 409 error.

Card 11: CloudEvents Protocol

Wraps pub/sub payloads inside CloudEvents schemas to preserve routing and tracing contexts across systems.

Card 15: Dead Letter Queue (DLQ)

Isolates poison messages in a dedicated Dead Letter Topic after maximum retry failures for offline diagnostics.

Card 4: Sidecar Process Proxy

Proxies outbound client calls and delivers subscribed events back to applications, handling reconnect buffering.

Card 8: State Transactions

Supports bulk transactional operations in a single gRPC/HTTP call, converting them to native DB transactions.

Card 12: Pluggable Brokers

Routes messages to Kafka, RabbitMQ, or Redis, shielding applications from custom consumer group setups.

Card 16: Virtual Actor Lifecycle

Activates stateful Actors dynamically in memory when called and deactivates idle ones to free up RAM.

Card 17: Actor Placement

Employs consistent hashing algorithms to register Actor locations and rebalance them if nodes fail.

Card 21: Input Bindings

Attaches to external event sources (databases, queues) and delivers events to applications via local POST calls.

Card 25: mTLS Mutual Auth

Encrypts traffic between Sidecar nodes using mutual TLS protocols, rotating certificates via Sentry automatically.

Card 18: Actor Consistency

Restricts execution to a single thread, queueing incoming requests and auto-loading/saving state to avoid locks.

Card 22: Output Bindings

Invokes external services (SendGrid, S3) via local POST calls, standardizing integrations without custom SDKs.

Card 26: SPIFFE Identity Mgmt

Formats identities via SPIFFE specs to enforce access control policies (ACLs) based on namespace attributes.

Card 19: Timers Reminders

Timers run callbacks during active Actor lifetimes; Reminders persist in storage to wake up actors when triggered.

Card 23: Declarative Config (YAML)

Configures bindings, pub/subs, and state stores inside YAML files, resolving passwords via secret store references.

Card 27: Circuit Breakers Retry

Implements declarative circuit breakers to trip connections if downstream error thresholds are exceeded.

Card 20: Reentrancy Deadlocks

Permits nested reentrant calls using a shared reentrancy ID, bypassing turn-based locking to prevent deadlocks.

Card 24: Bidirectional Stream

Supports persistent bidirectional data channels (WebSockets, gRPC stream) via local sidecars to lower latency.

Card 28: OpenTelemetry Tracking

Propagates trace context headers and exports spans across state, pub/sub, and bindings to OTEL backends.